DATABASE MANAGEMENT SYSTEM

Chapter 1: Introduction

CT 652 | BE BCT | Year III / Part II | TU IOE | Marks Weightage: 4

Syllabus Overview

1.1	1.1 Concepts and Applications
1.2	1.2 Objective and Evolution
1.3	1.3 Data Abstraction and Data Independence
1.4	1.4 Schema and Instances
1.5	1.5 Concepts of DDL, DML and DCL

1.1 Concepts and Applications

What is a Database?

A database is an organized collection of structured data stored electronically in a computer system, usually controlled by a DBMS.

What is DBMS?

A Database Management System (DBMS) is software that enables users to define, create, maintain, and control access to a database. It acts as an interface between end-users and the database.

DBMS vs File System

Exam Tip: "Why prefer DBMS over file system?" [4 marks] — Very frequently asked!

DBMS	File System
Reduces data redundancy	High data redundancy across files
Data consistency maintained	Inconsistency is common
Multi-user data sharing supported	Difficult to share data
Data integrity via constraints	No built-in integrity control
Better security and access control	Limited security mechanisms
Backup and recovery provided	Manual backup required
Concurrency control supported	Poor concurrency handling
Data independence achieved	No data independence

Applications of DBMS

- Banking: Account transactions, loan records, customer information
- Airlines: Reservations, scheduling, ticketing systems

- Universities: Registration, student grades, course management
- Healthcare: Patient records, appointments, billing
- E-commerce: Products, orders, customer management
- Telecommunications: Call records, billing, network management

1.2 Objective and Evolution

Objectives of DBMS

- Efficient data storage, retrieval and update
- Data sharing among multiple users and programs
- Transaction support with ACID properties
- Concurrency control — simultaneous access without conflicts
- Recovery services — restore data after failures
- Authorization and security controls
- Data independence — insulate programs from data representation changes

Evolution of Database Systems

Era / Generation	Characteristics
1950s-60s: File Systems	Flat files, high redundancy, no relationships
1960s-70s: Hierarchical Model	Tree structure (IBM IMS), parent-child relationships
1970s: Network Model	Graph structure, CODASYL standard, complex navigation
1970s-80s: Relational Model	E.F. Codd — SQL, tables, keys, joins (RDBMS)
1990s: Object-Oriented DB	OOP integrated — objects, inheritance, encapsulation
2000s-Present: NoSQL/NewSQL	Big data, distributed — MongoDB, Cassandra, etc.

1.3 Data Abstraction and Data Independence

Exam Tip: "Briefly explain the three levels of data abstraction in RDBMS. What is physical data independence?" [3+1] — MOST repeated question!

Three Levels of Data Abstraction (ANSI/SPARC Model)

To hide complexity from users, DBMS uses a three-level architecture. Each level hides lower-level details from the level above it.

Level	Description	Who Uses It
View Level (External) — Highest	Describes the part of DB relevant to a particular user. Different users may see different views. Hides the logical and physical levels.	End users, Application developers

Logical Level (Conceptual) — Middle	Describes WHAT data is stored and the relationships among them. Hides physical storage details.	DBAs (Database Administrators)
Physical Level (Internal) — Lowest	Describes HOW data is physically stored on disk — files, indexes, storage blocks, access paths.	System programmers, DBMS designers

Data Independence

Data independence is the ability to modify the schema at one level without affecting the schema at the next higher level.

1. Physical Data Independence

The ability to change the physical schema (how data is stored) without altering the logical schema or application programs. Example: Changing from sequential files to B-tree indexed storage should not affect the logical model.

2. Logical Data Independence

The ability to change the logical schema (add/remove tables or columns) without affecting the external schema or application programs. This is harder to achieve than physical data independence.

> **Exam Tip:** Physical data independence is EASIER to achieve than logical data independence. This distinction is frequently tested in exams.

1.4 Schema and Instances

> **Exam Tip:** "What do you mean by Schema and Instances? Briefly explain the different levels of data abstraction." [2+3] — repeatedly asked.

Schema (Intension)

A schema is the overall design or structure of the database — what tables exist, their columns, data types, constraints, and relationships. It is defined at design time and rarely changes.

- Also called the intension of the database
- Defined using DDL commands
- Example: CREATE TABLE Student (sid INT PRIMARY KEY, name VARCHAR(50), age INT);

Instance (Extension / State)

An instance is the actual data stored in the database at a particular point in time. It changes frequently as records are inserted, updated, or deleted.

- Also called the extension or state of the database
- Modified using DML commands
- Example: The current set of rows present in the Student table at a given moment

Schema	Instance
Defines the structure of the database	Actual data stored at a given moment
Changes rarely	Changes frequently (INSERT/UPDATE/DELETE)

Defined using DDL	Modified using DML
Like a class definition in OOP	Like an object/instance in OOP
Independent of data values	Dependent on current data values

1.5 Concepts of DDL, DML and DCL

Exam Tip: "Define database. Explain DDL, DML and DCL with examples." [2+4] — very commonly asked question.

DDL — Data Definition Language

DDL is used to define, modify, and delete the structure (schema) of database objects such as tables, views, and indexes. DDL statements are auto-committed (changes are permanent immediately).

Command	Purpose	Example
CREATE	Creates new database objects	CREATE TABLE Student (sid INT PRIMARY KEY, name VARCHAR(50));
ALTER	Modifies existing structure	ALTER TABLE Student ADD age INT;
DROP	Deletes objects permanently	DROP TABLE Student;
TRUNCATE	Removes all rows (keeps structure)	TRUNCATE TABLE Student;
RENAME	Renames a database object	RENAME TABLE Student TO Learner;

DML — Data Manipulation Language

DML is used to insert, update, delete, and retrieve data within the database. DML statements are NOT auto-committed and can be rolled back.

Command	Purpose	Example
SELECT	Retrieves data from tables	SELECT name, age FROM Student WHERE age > 20;
INSERT	Adds new rows to a table	INSERT INTO Student VALUES (1, 'Ram', 21);
UPDATE	Modifies existing rows	UPDATE Student SET age = 22 WHERE sid = 1;
DELETE	Removes specific rows	DELETE FROM Student WHERE sid = 1;

Types of DML:

- Procedural DML: User specifies WHAT data is needed AND HOW to get it (e.g., embedded SQL in C programs).
- Non-Procedural (Declarative) DML: User specifies only WHAT data is needed, not how to retrieve it (e.g., SQL SELECT).

DCL — Data Control Language

DCL is used to control access and permissions to data in the database.

Command	Purpose	Example
GRANT	Gives access privileges to a user	GRANT SELECT, INSERT ON Student TO user1;
REVOKE	Removes access privileges from a user	REVOKE INSERT ON Student FROM user1;

Exam Tip: TCL (Transaction Control Language) — COMMIT, ROLLBACK, SAVEPOINT — manages transactions and may be asked separately.

Summary: DDL vs DML vs DCL

Language	Commands and Purpose
DDL — Data Definition Language	CREATE, ALTER, DROP, TRUNCATE — defines DB structure (auto-committed)
DML — Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE — manipulates data (can be rolled back)
DCL — Data Control Language	GRANT, REVOKE — controls access permissions
TCL — Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT — manages transactions

Past Exam Questions — Chapter 1

Exam Tip: Chapter 1 typically carries 3-4 marks in TU exams. Study all questions below.

Q: Briefly explain the three levels of data abstraction in RDBMS. What is physical data independence?	**[3+1]**
Q: What do you mean by Schema and Instances? Briefly explain the different levels of data abstraction.	**[2+3]**
Q: Define database and database management system. List advantages of DBMS over file management.	**[1+3]**
Q: Why is database system more preferred over file system? Explain briefly.	**[4]**
Q: Why do we prefer DBMS over file system for handling databases? Briefly explain.	**[4]**

Q: Define database language. Compare the DBMS with file processing system.	**[2+3]**
Q: Define database. Explain DDL, DML and DCL with examples.	**[2+4]**
Q: Define the terms Data Abstraction and Data Independence. Why are they important in DBMS?	**[2+3]**
Q: What do you mean by data abstraction? List the various levels of data abstraction and briefly explain.	**[1+3]**
Q: Explain the layers of abstraction in database design. Why is physical data independence important in data modeling?	**[2+2]**
Q: Differentiate between schema and instances. What are the disadvantages of conventional file system?	**[4]**
Q: Distinguish between a database and a DBMS. What is the advantage of separating the logical and physical level in database design?	**[2+2]**
Q: What are the drawbacks of file system to store data?	**[4]**
Q: What difficulties would you face if you used file system directly to implement a database application? What is physical data independence?	**[3+1]**

Quick Revision — Key Points

- Database = organized collection of related data stored electronically.
- DBMS = software to define, create, maintain and control database access.
- 3 Levels: Physical (HOW stored) -> Logical (WHAT stored) -> View (user perspective).
- Physical Data Independence = change storage without affecting logical schema (EASIER to achieve).
- Logical Data Independence = change logical schema without affecting user views (HARDER to achieve).
- Schema = structure/design of DB (changes rarely) — the intension.
- Instance = actual data in DB at a moment (changes often) — the extension.
- DDL commands: CREATE, ALTER, DROP, TRUNCATE — define structure (auto-committed).
- DML commands: SELECT, INSERT, UPDATE, DELETE — manipulate data (can rollback).
- DCL commands: GRANT, REVOKE — control access permissions.
- DBMS advantages over file system: no redundancy, consistency, integrity, concurrency, security, independence, backup/recovery.